

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME: Mukesh Raj L

REG NO: 192571036

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

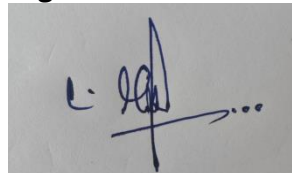
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5

7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I Mukesh Raj L (192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding ; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools

Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and	Organized documentation with necessary	Basic documentation with missing details	Poor or incomplete documentation
		results	details		

1)

1. Functional Dependencies

Based on the meaning of the attributes:

Student ID determines Student Name

$SID \rightarrow SName$

A particular SID identifies one student, so it determines the student's name.

Course ID determines Course Name and Instructor

$CID \rightarrow CName, Instructor$

A particular CID identifies one course, so it determines the course name and instructor.

Student-Course combination determines Grade

$SID, CID \rightarrow Grade$

A student's grade depends on **which course** the student takes.

Therefore, the main functional dependencies are:

$SID \rightarrow SName$

$CID \rightarrow CName$

$CID \rightarrow Instructor$

$SID, CID \rightarrow Grade$

They can also be written together as:

$SID \rightarrow SName$

$CID \rightarrow CName, Instructor$

$SID \text{ } CID \rightarrow Grade$

2. Determine the Candidate Key

We need to find an attribute set that determines **all attributes**.

Consider:

$\{SID, CID\}$

Its closure is:

$(SID, CID)^+$

From:

$SID \rightarrow SName$

we get:

$SName$

From:

$CID \rightarrow CName, Instructor$

we get:

CName, Instructor

From:

SID, CID $\rightarrow$ Grade

we get:

Grade

Therefore:

(SID, CID)⁺
= {SID, CID, SName, CName, Instructor, Grade}

This contains **all attributes** of the relation.

Therefore:

{SID, CID}

is a **superkey**.

Neither SID nor CID alone can determine all attributes, so the combination is minimal.

Hence, the **candidate key** is:

(SID, CID)

2)

Step 1: Unnormalized Relation (UNF)

Consider a student relation where a student can have **multiple courses** stored in a single field.

STUDENT(
 SID,
 SName,
 Courses
)

Example:

SID	SName	Courses
101	Arun	DBMS, OS
102	Bala	DBMS, CN

Here, the Courses attribute contains **multiple values** in a single cell.

Therefore, the relation is **not in 1NF**.

Step 2: Identify the Problem

The main problem is:

Courses = DBMS, OS

This is a **multi-valued attribute**.

In **First Normal Form (1NF)**, every attribute must contain **only atomic (single) values**.

Therefore, we must separate the multiple course values into individual rows.

Step 3: Convert to 1NF

Create one row for each student-course combination.

1NF Relation

STUDENT_COURSE(SID, SName, Course)

SID	SName	Course
101	Arun	DBMS
101	Arun	OS
102	Bala	DBMS
102	Bala	CN

Now, every cell contains a **single atomic value**.

Step 4: Identify the Key

A student can take multiple courses, so SID alone is not sufficient to uniquely identify a row.

The combination:

(SID, Course)

uniquely identifies each student-course record.

Therefore:

Candidate Key = (SID, Course)

Step 5: Justification for 1NF

The relation is now in **1NF** because:

1. Each attribute contains atomic values.
2. There are no repeating groups.
3. There are no multi-valued attributes within a single cell.
4. Each row represents one student-course relationship.
5. (SID, Course) uniquely identifies each record.

3)

Step 1: Given Relation

$R(A, B, C, D, E)$

Functional dependencies:

$A \rightarrow BC$

$BC \rightarrow D$

$D \rightarrow E$

Step 2: Find the Candidate Key

We calculate the closure of A.

$A^+ = \{A\}$

Using:

$A \rightarrow BC$

we get:

$A^+ = \{A, B, C\}$

Using:

$BC \rightarrow D$

we get:

$A^+ = \{A, B, C, D\}$

Using:

$D \rightarrow E$

we get:

$A^+ = \{A, B, C, D, E\}$

Therefore:

$A^+ = \{A, B, C, D, E\}$

So A determines all attributes.

Hence:

Candidate Key = A

Step 3: Check for Partial Dependency

A **partial dependency** occurs when a non-prime attribute depends on only part of a composite candidate key.

Here, the candidate key is only:

A

It is a **single attribute**, not a composite key.

Therefore, **partial dependency does not exist**.

So the relation is already in **2NF**.

Step 4: Decomposition

Since there is no partial dependency, **no decomposition is required to achieve 2NF**.

Therefore:

$R(A, B, C, D, E)$

remains the 2NF relation.

However, the FD:

$D \rightarrow E$

creates a **transitive dependency**, which is relevant when converting the relation to **3NF**, not 2NF.

4)

Given relation:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Functional dependencies:

$EmpID \rightarrow EmpName, DeptID$

$DeptID \rightarrow DeptName, ManagerID$

Step 1: Identify the Transitive Dependency

$EmpID \rightarrow DeptID$

$DeptID \rightarrow DeptName, ManagerID$

Therefore:

$EmpID \rightarrow DeptName, ManagerID$

This is a **transitive dependency**.

Step 2: Decompose into 3NF

Relation 1:

EMPLOYEE(EmpID, EmpName, DeptID)

Primary Key:

EmpID

Relation 2:

DEPARTMENT(DeptID, DeptName, ManagerID)

Primary Key:

DeptID

Result

EMPLOYEE(EmpID, EmpName, DeptID)

DEPARTMENT(DeptID, DeptName, ManagerID)

The transitive dependency is removed, and the relations are in **3NF**.

5)

Given:

R(A, B, C, D)

Functional dependencies:

$AB \rightarrow C$

$C \rightarrow D$

$D \rightarrow A$

Step 1: Find Candidate Keys

AB⁺:

$AB \rightarrow C$

$C \rightarrow D$

$D \rightarrow A$

$AB^+ = \{A, B, C, D\}$

So, AB is a candidate key.

BC⁺:

$C \rightarrow D$

$D \rightarrow A$

$BC^+ = \{A, B, C, D\}$

So, BC is a candidate key.

BD⁺:

$D \rightarrow A$

$AB \rightarrow C$

$BD^+ = \{A, B, C, D\}$

So, BD is a candidate key.

Candidate keys:

AB, BC, BD

Step 2: Check BCNF

For BCNF, for every non-trivial FD $X \rightarrow Y$, **X must be a superkey**.

Check:

$AB \rightarrow C$

AB is a candidate key $\rightarrow$ satisfies BCNF.

$C \rightarrow D$

C is **not** a superkey $\rightarrow$ violates BCNF.

$D \rightarrow A$

D is **not** a superkey $\rightarrow$ violates BCNF.

Therefore:

R is NOT in BCNF.

Step 3: Decompose using $C \rightarrow D$

Create:

$R_1(C, D)$

$R_2(A, B, C)$

For R_1 :

$C \rightarrow D$

C is the key of R_1 , so R_1 is in BCNF.

For R_2 :

$AB \rightarrow C$

AB is a key of R_2 , so R_2 is in BCNF.

Therefore, the BCNF decomposition is:

$R_1(C, D)$

$R_2(A, B, C)$

The dependency $D \rightarrow A$ is not preserved in this decomposition, but the decomposition is in **BCNF**.

6)

1. Introduction

A decomposition of a relation is said to be **lossless-join** if joining the decomposed relations produces exactly the original relation without generating any spurious tuples or losing information.

The **Chase algorithm** is a tabular method used to verify whether a decomposition is lossless with respect to a given set of functional dependencies.

2. Given Relation

Consider the relation:

$R(A, B, C)$

Suppose it is decomposed into:

$R_1(A, B)$

$R_2(B, C)$

The functional dependency is:

$B \rightarrow C$

We need to verify whether this decomposition is **lossless**.

3. Construct the Chase Table

Create a table with one row for each decomposed relation and one column for every attribute of the original relation.

Use distinguished symbols for attributes present in each relation.

Relation	A	B	C
$R_1(A, B)$	a	b	c1
$R_2(B, C)$	a2	b	c

Here:

- a, b, and c are distinguished symbols.
- a2 and c1 are different symbols.
- Since R_1 contains A and B, their values are represented by the distinguished symbols.
- Since R_2 contains B and C, their values are represented by the distinguished symbols.

4. Apply the Functional Dependency

The given functional dependency is:

$B \rightarrow C$

Observe the Chase table:

R1: A = a B = b C = c1

R2: A = a2 B = b C = c

Both rows have the **same value b for attribute B**.

According to the functional dependency:

$B \rightarrow C$

if two tuples have the same value of B, they must have the same value of C.

Therefore, the value of C in the first row is changed to the distinguished value c.

5. Updated Chase Table

Relation	A	B	C
R1(A,B)	a	b	c
R2(B,C)	a2	b	c

Now the first row contains the distinguished symbols for **all attributes**:

A = a

B = b

C = c

Therefore, the Chase algorithm succeeds.

6. Lossless-Join Verification

The condition for a lossless decomposition is satisfied because the Chase table contains a row consisting entirely of distinguished symbols.

Therefore:

$R(A,B,C) = R1(A,B) \bowtie R2(B,C)$

Hence, the decomposition is **lossless-join**.

7. Why is Lossless Join Important?

A lossless decomposition ensures that:

- No information is lost during decomposition.
- Joining the decomposed relations reconstructs the original relation.
- No unwanted or spurious tuples are generated.
- Database redundancy can be reduced without losing information.

7)

1. Given Relation

TEACHER(TID, Subject, Hobby)

Where:

- TID = Teacher ID
- Subject = Subject taught by the teacher
- Hobby = Hobby of the teacher

A teacher can teach **multiple subjects** and can have **multiple hobbies**.

These two sets of values are independent of each other.

2. Multi-Valued Dependencies

The following multi-valued dependencies exist:

$TID \twoheadrightarrow Subject$

This means that for a particular teacher, there can be multiple subjects independently.

Also:

$TID \twoheadrightarrow Hobby$

This means that for a particular teacher, there can be multiple hobbies independently.

3. Example of Redundancy

Suppose teacher T1 teaches:

DBMS
Operating Systems

and has hobbies:

Cricket
Music

The original relation must store all combinations:

TID	Subject	Hobby
T1	DBMS	Cricket
T1	DBMS	Music
T1	Operating Systems	Cricket
T1	Operating Systems	Music

This creates unnecessary redundancy because the subjects and hobbies are independent.

4. Why It Violates 4NF

A relation is in **Fourth Normal Form (4NF)** if, for every non-trivial multivalued dependency:

$$X \twoheadrightarrow Y$$

X must be a **superkey**.

Here:

$$TID \twoheadrightarrow \text{Subject}$$

$$TID \twoheadrightarrow \text{Hobby}$$

But TID alone is not a superkey of:

TEACHER(TID, Subject, Hobby)

Therefore, the relation violates **4NF**.

5. Decomposition into 4NF

Separate the independent multi-valued attributes into two relations.

Relation 1

TEACHER_SUBJECT(TID, Subject)

Relation 2

TEACHER_HOBBY(TID, Hobby)

6. Resulting Relations

TEACHER_SUBJECT

TID	Subject
T1	DBMS
T1	Operating Systems

TEACHER_HOBBY

TID	Hobby
T1	Cricket
T1	Music

Now the independent multi-valued information is stored separately.

7. 4NF Verification

For:

TEACHER_SUBJECT(TID, Subject)

$TID \twoheadrightarrow \text{Subject}$ is no longer a problematic dependency because TID, Subject forms the relation's key.

For:

TEACHER_HOBBY(TID, Hobby)

TID, Hobby forms the key.

Thus, the decomposition removes the non-trivial multivalued dependencies that violated 4NF.

8)

1. Given Relation

Consider:

SUPPLY(Supplier, Part, Project)

A supplier can supply a particular part to a particular project.

Suppose the business rules indicate that these three relationships are independent:

- Supplier $\leftrightarrow$ Part
- Supplier $\leftrightarrow$ Project
- Part $\leftrightarrow$ Project

This creates a **join dependency**.

2. Example of Redundancy

Suppose:

Supplier = S1

Parts = P1, P2

Projects = J1, J2

The relation may contain:

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

The same supplier-part and supplier-project information is repeated across multiple tuples.

This causes **redundancy**.

3. Join Dependency

The relation has a join dependency:

$SUPPLY = \pi_{Supplier, Part}(SUPPLY)$

- ⋈ $\pi_{\text{Supplier,Project}}(\text{SUPPLY})$
- ⋈ $\pi_{\text{Part,Project}}(\text{SUPPLY})$

This means the original relation can be reconstructed by joining three projections.

4. Decomposition into 5NF

Decompose the relation into three relations:

SUPPLIER_PART

SUPPLIER_PART(Supplier, Part)

SUPPLIER_PROJECT

SUPPLIER_PROJECT(Supplier, Project)

PART_PROJECT

PART_PROJECT(Part, Project)

For the example:

SUPPLIER_PART

Supplier	Part
S1	P1
S1	P2

SUPPLIER_PROJECT

Supplier	Project
S1	J1
S1	J2

PART_PROJECT

Part	Project
P1	J1
P1	J2
P2	J1
P2	J2

5. Why This is 5NF

Fifth Normal Form (5NF) removes redundancy caused by **join dependencies**.

The original relation contains a three-way relationship and can be reconstructed from smaller relations.

After decomposition:

SUPPLIER_PART



SUPPLIER_PROJECT

↓
PART_PROJECT

The independent relationships are stored separately, reducing redundancy.

The original relation can be reconstructed using:

SUPPLIER_PART
⋈
SUPPLIER_PROJECT
⋈
PART_PROJECT

Thus, the decomposition is designed to satisfy **5NF / Project-Join Normal Form**.

9)

1. Poorly Designed Relation

Consider:

COLLEGE(
 StudentID,
 StudentName,
 CourseID,
 CourseName,
 InstructorID,
 InstructorName
)

Example:

StudentID	StudentName	CourseID	CourseName	InstructorID	InstructorName
S1	Arun	C1	DBMS	I1	Kumar
S2	Bala	C1	DBMS	I1	Kumar
S3	Charan	C2	OS	I2	Priya

2. Functional Dependencies

StudentID → StudentName

CourseID → CourseName, InstructorID

InstructorID → InstructorName

Therefore:

CourseID → InstructorID → InstructorName

This creates a **transitive dependency**.

3. Insertion Anomaly

Suppose a new course is introduced:

C3 = Computer Networks

but no student has enrolled yet.

In the original table, we may be unable to insert the course information because StudentID is required.

Therefore, the database has an **insertion anomaly**.

4. Deletion Anomaly

Suppose student S3 is the only student enrolled in course C2.

If we delete student S3, we may also lose:

C2 = OS

I2 = Priya

So deleting a student unintentionally removes course and instructor information.

This is a **deletion anomaly**.

5. Update Anomaly

Suppose instructor I1 changes their name from:

Kumar → Dr. Kumar

The name must be changed in every row containing I1.

If only some rows are updated, inconsistent data will exist.

This is an **update anomaly**.

6. Normalize to 1NF

The relation already contains atomic values, so it satisfies **1NF**.

```
COLLEGE(  
    StudentID,  
    StudentName,  
    CourseID,  
    CourseName,  
    InstructorID,  
    InstructorName  
)
```

7. Convert to 2NF

The candidate key is:

(StudentID, CourseID)

There are partial dependencies:

$\text{StudentID} \rightarrow \text{StudentName}$

$\text{CourseID} \rightarrow \text{CourseName}, \text{InstructorID}$

Therefore, decompose into:

STUDENT

STUDENT(StudentID, StudentName)

COURSE

COURSE(CourseID, CourseName, InstructorID, InstructorName)

ENROLLMENT

ENROLLMENT(StudentID, CourseID)

8. Convert to 3NF

In COURSE:

$\text{CourseID} \rightarrow \text{InstructorID}$

$\text{InstructorID} \rightarrow \text{InstructorName}$

Therefore:

$\text{CourseID} \rightarrow \text{InstructorName}$

This is a **transitive dependency**.

So separate the instructor information.

STUDENT

STUDENT(StudentID, StudentName)

COURSE

COURSE(CourseID, CourseName, InstructorID)

INSTRUCTOR

INSTRUCTOR(InstructorID, InstructorName)

ENROLLMENT

ENROLLMENT(StudentID, CourseID)

9. Final 3NF Schema

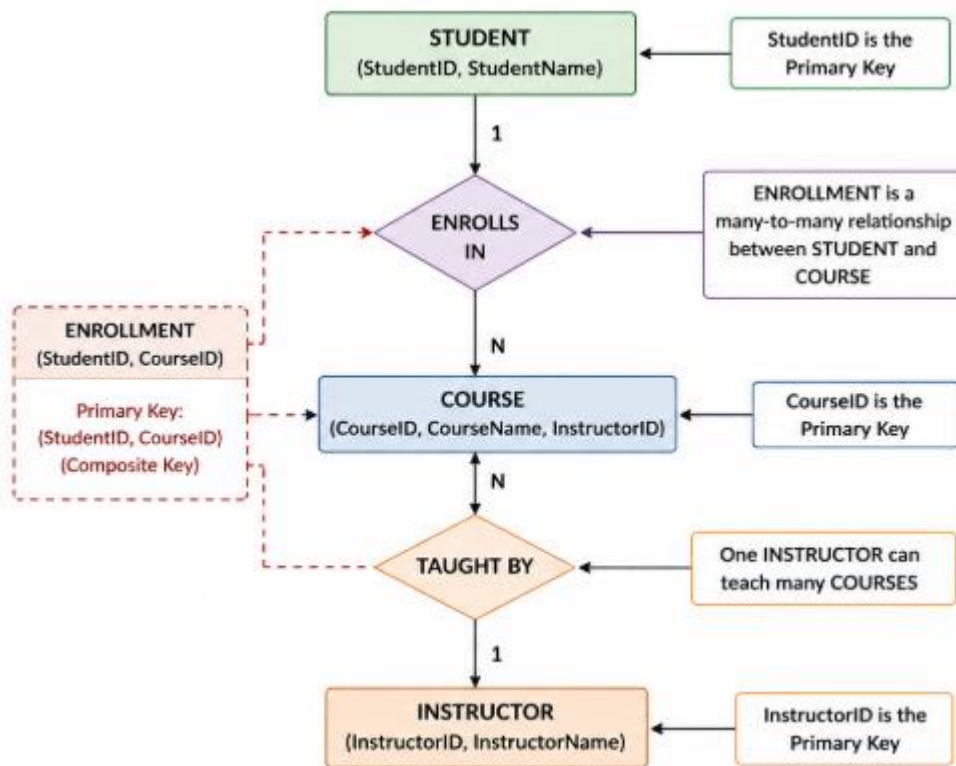
STUDENT(StudentID, StudentName)

COURSE(CourseID, CourseName, InstructorID)

INSTRUCTOR(InstructorID, InstructorName)

ENROLLMENT(StudentID, CourseID)

Relationships



This removes the major insertion, deletion, and update anomalies by storing each piece of information in the appropriate relation.

10)

Definition

The **closure of an attribute set X**, denoted by X^+ , is the set of all attributes that can be functionally determined by X using the given functional dependencies.

A **candidate key** is a minimal set of attributes whose closure contains **all attributes of the relation**.

Given Relation

Consider:

$R(A, B, C, D, E)$

Functional dependencies:

$A \rightarrow BC$
 $BC \rightarrow D$
 $D \rightarrow E$

Step 1: Calculate A^+

Initially, the closure contains only A.

$A^+ = \{A\}$

From the functional dependency:

$A \rightarrow BC$

we can add B and C.

$A^+ = \{A, B, C\}$

Now we have both B and C.

Therefore, we can apply:

$BC \rightarrow D$

and add D.

$A^+ = \{A, B, C, D\}$

Now we have D, so we can apply:

$D \rightarrow E$

and add E.

$A^+ = \{A, B, C, D, E\}$

Therefore:

$A^+ = \{A, B, C, D, E\}$

Since A^+ contains **all attributes of R**, A is a **superkey**.

Because A consists of only one attribute, it is minimal.

Therefore:

A is a candidate key.

Step 2: Check Whether B Can Be a Candidate Key

Start with:

$B^+ = \{B\}$

There is no FD beginning with only B.

Therefore:

$B^+ = \{B\}$

It does not contain all attributes.

Hence:

B is not a candidate key.

Step 3: Check C

Start with:

$$C^+ = \{C\}$$

There is no FD beginning with only C.

Therefore:

$$C^+ = \{C\}$$

So C cannot determine all attributes.

Hence:

C is not a candidate key.

Step 4: Check D

Start with:

$$D^+ = \{D\}$$

Using:

$$D \rightarrow E$$

we get:

$$D^+ = \{D, E\}$$

No other FD can be applied.

Therefore:

$$D^+ = \{D, E\}$$

Since it does not contain all attributes:

D is not a candidate key.

Step 5: Check Other Possible Combinations

Since A alone already determines every attribute:

$$A^+ = \{A, B, C, D, E\}$$

any combination containing A, such as:

AB

AC

AD

AE

ABC

will be a **superkey**, but none will be a candidate key because they are not minimal.

For example:

$(AB)^+ = \{A, B, C, D, E\}$

But A alone already determines everything, so AB is not minimal.

Step 6: Candidate Key

Therefore, the minimal attribute set that determines all attributes is:

Candidate Key = $\{A\}$